Pontificia Universidad JAVERIANA Colombia

MCP: conectar agentes y datos

Semana 7 · Construir y consumir servidores MCP
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

Fundamentos con LangChain
Semanas 1–3 ✓

RAG y evaluación
Semanas 4–6 ✓

Model Context Protocol
Semana 7

LangGraph y patrones agénticos
Semanas 8–11

LLMOps y despliegue
Semana 12

Semana 7 · Model Context Protocol

- Una integración para varias aplicaciones
- Host, cliente, servidor y modelo
- Tools, resources y prompts
- Un servidor de créditos sobre SQLite
- Un agente con dos servidores
- stdio, HTTP, Claude Code y seguridad

Las ideas que traemos de la semana 6

Repaso

Antes de avanzar

1. Una evaluación compara configuraciones con la misma entrada.
2. Las comprobaciones de código van primero; el juez se calibra.
3. Una cita no basta: hay que comprobar que viene de lo recuperado.

Una consulta, varias aplicaciones

El problema de la semana

¿Cómo compartimos la herramienta?

La consulta de la cartera vive en un notebook (`@tool`).

El asistente de soporte, Claude Code y otra aplicación del equipo necesitan **la misma consulta**.

¿Copiamos el código en cada una?

Qué es MCP

Model Context Protocol

Un protocolo abierto

Un contrato común para **descubrir** y **usar** herramientas, datos y plantillas de otros programas.

Un **servidor MCP** es el programa que ofrece esas capacidades. «Conectar un MCP» suele significar conectar un servidor.

Quién hace cada parte

Responsabilidades de la integración

Componente	Responsabilidad	En el laboratorio
Host	Coordina modelo, clientes y permisos	La aplicación del notebook
Cliente MCP	Habla el protocolo con un servidor	`Client` del SDK o `MCPAdapter`
Servidor MCP	Expone y ejecuta capacidades	`credits.py` y `documents.py`
Modelo	Propone tools y redacta la respuesta	Claude Haiku

Las tres primitivas del servidor

Quién dirige su uso

Primitiva	Quién la elige	En ACME
Tool	El modelo propone la operación	`check_credit`, `search_documents`
Resource	La aplicación elige qué leer, por URI	`schema://credits`, `docs://{name}`
Prompt	La persona elige una plantilla	«Solicitud de reestructuración»

El recorrido de una consulta

Después del descubrimiento (tools/list)



El modelo propone; la aplicación ejecuta; el servidor implementa la operación.

Pontificia Universidad JAVERIANA Colombia

Construir el servidor

L12-MCPServer · sin modelo

Una tool en un servidor MCP

src/agents/servers/credits.py

El contrato

Tipos y descripciones forman el esquema que ve el modelo.

```
from mcp.server import MCPServer

mcp = MCPServer("creditos")

@mcp.tool(annotations=READ_ONLY)
async def check_credit(customer: Customer,
                       choice: ...,  # elicitation
                       product: Product = "") -> list[CreditRow]:
    """Consulta los créditos de un cliente de ACME.

    Devuelve una fila por crédito con el saldo, el monto
    vencido y los días de mora, en pesos colombianos.
    """
    rows = find_credits(customer, product)
    ...
```

Hablar MCP sin modelo

L12 · secciones 3 a 5

Descubrir y llamar

Probar el servidor
no gasta tokens.

```
from mcp import Client

async with Client(mcp) as client:
    tools = await client.list_tools()
    result = await client.call_tool(
        "check_credit",
        {"customer": "C-3307", "product": "CLI"})
    schema = await client.read_resource("schema://credits")

result.structured_content
# overdue 2.070.000 COP · 75 días de mora
```

Validar y fallar bien

L12 · sección 4

Controles en el código

1. La conexión SQLite se abre en modo solo lectura (`mode=ro`).
2. La consulta usa parámetros; nunca concatena la entrada del usuario.
3. Un código de cliente inválido produce un `ToolError` claro.
4. `read_only_hint` informa al cliente, pero no impide una escritura: eso lo hace el código.

Preguntar a la persona: elicitation

L12 · sección 7

Una pregunta ambigua

1. «Soy el cliente C-1042. ¿Cuánto tengo vencido?»: tiene un CEX y un CLI.
2. El servidor pide a la persona que elija el producto.
3. La respuesta vuelve al servidor y la consulta continúa.

Seguridad

L12 · sección 9

Cada servidor añade riesgo

1. Conceder solo el acceso necesario a datos y operaciones.
2. **Tool poisoning**: la descripción de una tool también es una instrucción para el modelo.
3. Revisar el origen, los permisos y las descripciones antes de conectar un servidor ajeno.
4. Separar claves y registros del canal del protocolo: un servidor stdio no escribe en stdout.

Pontificia Universidad JAVERIANA Colombia

Consumir los servidores

L13-MCPAgent

Un agente con dos servidores

L13 · secciones 1 y 2

MCPAdapter

El agente de la semana 3, con tools que viven fuera.

```
from langchain.mcp import MCPAdapter

servers = {"mcpServers": {
    "creditos": {"command": sys.executable,
                 "args": ["-m", "agents.servers.credits"]},
    "documentos": {"command": sys.executable,
                   "args": ["-m", "agents.servers.documents"]},
}}
async with MCPAdapter(servers) as adapter:
    tools = await adapter.list_tools()
agent = create_agent(model, tools=tools,
                     system_prompt=SYSTEM_PROMPT)
```

Qué servidor respondió

L13 · sección 3 · cliente C-3307

Llamada	Evidencia
`creditos → check_credit`	Vencido 2.070.000 COP · 75 días de mora
`documentos → search_documents`	Reestructuración con hasta 90 días de mora [acme_collections_policy.pdf, p. 2]

stdio y Streamable HTTP

L13 · secciones 5 y 6

Aspecto	stdio	HTTP
Conexión	Entrada y salida de un proceso	Una URL del servidor
Arranque	Lo gestiona el cliente	Lo iniciamos nosotros
Contrato	`search_documents(query)`	`search_documents(query)`
Claude Code	`.mcp.json` registra los dos servidores	`/mcp` revisa las conexiones

Muchos servidores, muchas herramientas

L13 · secciones 7 y 8

Descubrimiento

1. El MCP Registry ayuda a encontrar servidores publicados, no garantiza que sean seguros.
2. MCP Inspector permite probar un servidor sin escribir código.
3. Con muchas tools, el descubrimiento progresivo evita llenar el contexto.

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L12-MCPServer · L13-MCPAgent

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. L12: construir el servidor de créditos y probarlo con el cliente, sin modelo.
2. L12: resources, elicitation y el servidor de documentos.
3. L13: un agente con los dos servidores (unas 7 llamadas).
4. L13: elicitation desde el agente, HTTP y Claude Code.

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. MCP separa quién implementa una herramienta de quién la usa.
2. Tool, resource y prompt se diferencian por quién decide usarlos.
3. El transporte cambia; el contrato se mantiene.
4. Los controles de seguridad están en el código del servidor.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
modelcontextprotocol.io	Arquitectura, primitivas y transportes
SDK de Python de MCP	`MCPServer`, `Client` y elicitation
Documentación de LangChain	`langchain.mcp` y `MCPAdapter`

Pontificia Universidad JAVERIANA Colombia

Próxima semana

LangGraph: estado, nodos, aristas y los primeros patrones de flujo.
Notebooks:
L14 a L19

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso